

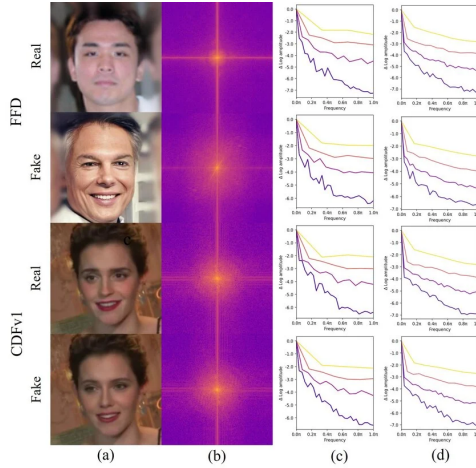
# FauxTo: Rilevamento Deepfake con Analisi FTT

Elena Giunta

## 1 Introduzione

L'obiettivo del progetto è sviluppare un'applicazione Android in grado di determinare se un'immagine è autentica o generata da intelligenza artificiale, analizzando lo spettro delle frequenze ottenuto tramite FFT (*Fast Fourier Transform*).

Il principio alla base del sistema è che le immagini generate tramite AI presentano artefatti nel dominio delle frequenze che sono del tutto assenti nelle fotografie reali. Tali artefatti, introdotti dalle operazioni di upsampling impiegate dalle reti generative, risultano visibili nello spettro FFT sotto forma di pattern anomali, particolarmente evidenti nei quadranti dell'immagine.



L'implementazione ha richiesto di affrontare diverse complessità tecniche:

- **Assenza di modelli pre-addestrati:** Non esistono modelli ONNX<sup>1</sup> già pronti per questo specifico compito, rendendo necessario l'addestramento di una rete neurale da zero.
- **Addestramento del modello:** Per l'analisi è stato impiegato un modello di deep learning con architettura dual-branch, che esamina contemporaneamente l'immagine originale e il relativo spettro delle frequenze, combinandone le feature estratte per la classificazione finale.
- **Conversione e integrazione:** Il modello, inizialmente addestrato in TensorFlow<sup>2</sup>, è stato convertito in formato ONNX per l'esecuzione su dispositivi Android tramite ONNX Runtime, consentendo così un'analisi completamente offline.

Il risultato di questo lavoro è **FauxTo**, un'applicazione Android progettata per verificare l'autenticità delle immagini. Il sistema permette all'utente di sottoporre ad analisi in locale immagini, senza necessità di connessione internet, attraverso l'esame del suo spettro delle frequenze. Al termine del processo, l'applicazione fornisce un verdetto immediato sotto forma di percentuale di probabilità, offrendo uno strumento concreto per identificare contenuti manipolati o generati artificialmente.

### 1.1 Specifiche Tecniche e Architettura Applicazione

L'applicazione è stata sviluppata in ambiente Android Studio, adottando le seguenti tecnologie:

<sup>1</sup>Open Neural Network Exchange (ONNX) è un formato open-source creato per facilitare l'intercambiabilità dei modelli di machine learning tra diverse piattaforme e strumenti.

<sup>2</sup>TensorFlow è una libreria open-source per il calcolo numerico, il machine learning su larga scala, il deep learning e altre attività di analisi statistica e predittiva.

- **Requisiti minimi:** API 24 (Android 7.0), garantendo la compatibilità con la maggior parte dei dispositivi Android moderni.
- **Linguaggio:** Java.
- **Persistenza Dati:** SQLite tramite `SQLiteOpenHelper` per la gestione delle credenziali utente.
- **Elaborazione Immagini:** Libreria *JTransforms* per l'implementazione efficiente della FFT.
- **Machine Learning:** *ONNX Runtime* per l'esecuzione del modello neurale dual-branch direttamente sul dispositivo.

Il passaggio da un'activity all'altra avviene tramite l'utilizzo di Intent; i dati necessari (come l'immagine elaborata e l'esito dell'analisi) vengono trasferiti tra le schermate in modo da garantire la fluidità e la continuità del flusso applicativo.

L'applicazione si articola in quattro Activity principali che guidano l'utente attraverso un percorso lineare:

1. **LoginActivity:** gestisce l'autenticazione (login) dell'utente.
2. **RegisterActivity:** gestisce la creazione di un nuovo account.
3. **ImageViewer:** rappresenta la schermata principale, dove l'utente carica l'immagine e avvia l'analisi.
4. **ResultActivity:** mostra l'esito finale dell'elaborazione, presentando la percentuale di classificazione e l'anteprima dello spettro FFT.

## 2 Gestione del Database

L'implementazione del sistema di autenticazione si basa su SQLite, un database relazionale locale. Questa scelta risulta perfettamente coerente con l'architettura di FauxTo: essendo un'applicazione progettata per eseguire l'analisi dello spettro delle frequenze (FFT) interamente on-device, deve garantire il pieno funzionamento anche in assenza di connettività internet. Trattandosi di uno strumento privo di funzionalità social o di condivisione in rete, l'integrazione di un database in cloud (come Firebase) avrebbe comportato una complessità architetturale superflua. SQLite permette di soddisfare il requisito di autenticazione mantenendo l'applicazione leggera, sicura e fedele al suo paradigma 100% offline.

### 2.1 Struttura del database

La gestione dei dati è affidata alla classe `DatabaseHelper`, che estende `SQLiteOpenHelper`, un componente base del framework Android utile a facilitare la creazione e la gestione dei database SQLite. Quest'ultimo è un motore relazionale leggero, nativamente integrato in Android, che non richiede installazioni o configurazioni aggiuntive. Il database `FauxTo.db` contiene un'unica tabella denominata `users`, destinata alla memorizzazione delle credenziali degli utenti registrati. La tabella si compone di tre colonne:

- **id:** chiave primaria di tipo `INTEGER` con autoincremento. Viene generata automaticamente a ogni inserimento per garantire l'univocità di ciascun record.
- **username:** campo di tipo `TEXT` che memorizza il nome utente scelto in fase di registrazione.
- **password:** campo di tipo `TEXT` che memorizza la password associata all'account.

La creazione della tabella avviene nel metodo `onCreate()`, eseguito automaticamente alla prima apertura dell'applicazione, tramite il comando SQL:

```
CREATE TABLE users (id INTEGER PRIMARY KEY AUTOINCREMENT,
username TEXT, password TEXT)
```

La classe `DatabaseHelper` espone inoltre tre metodi pubblici, ciascuno con uno scopo specifico per la gestione degli utenti:

- **addUser(user, pass):**  
inserisce un nuovo record nella tabella `users`. Utilizza un oggetto `ContentValues` per associare i valori alle rispettive colonne e `getWritableDatabase()` per ottenere l'accesso in modalità scrittura. Al termine, la connessione viene chiusa con `close()`.

- **checkUserExists(user)**: verifica se un nome utente è già presente nel database per prevenire la creazione di account duplicati. Esegue una query **WHERE** e restituisce **true** se il cursore contiene almeno un risultato. Viene invocato durante la registrazione per impedire la creazione di account duplicati.
- **checkUser(user, pass)**: controlla la validità delle credenziali durante il login. Restituisce **true** solo se la query a doppia condizione (username e password) trova un record esattamente corrispondente.

Le operazioni di sola lettura impiegano **getReadableDatabase()**, mentre i risultati delle query vengono gestiti sotto forma di oggetto **Cursor**, un iteratore che scorre le righe della tabella. Per le verifiche di esistenza è sufficiente controllare che il metodo **getCount()** restituisca un valore maggiore di zero.

## 2.2 Gestione delle risorse

Al termine di ogni operazione, sia il **Cursor** che il **SQLiteDatabase** vengono chiusi esplicitamente tramite il metodo **close()**. In Android, questa pratica è fondamentale per evitare memory leak e garantire la corretta liberazione delle risorse di sistema.

## 2.3 Aggiornamento del database e persistenza delle credenziali

Il metodo **onUpgrade()** gestisce eventuali cambi di versione della struttura del database. Nella versione attuale, in caso di aggiornamento, la tabella esistente viene eliminata e ricreata. Poiché il database risiede nella memoria interna, i dati sono persistenti tra i riavvii dell'app ma restano vincolati al singolo dispositivo; di conseguenza, le credenziali create su uno smartphone non saranno accessibili su altri device.

## 2.4 Hashing delle password

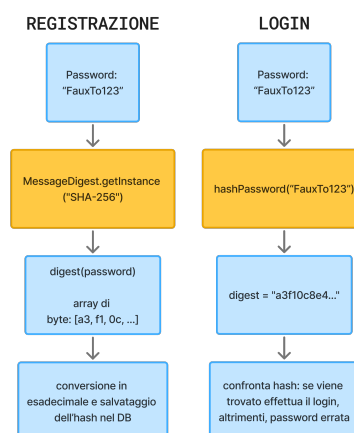
Per garantire la protezione dei dati sensibili, l'applicazione evita accuratamente di memorizzare le password in chiaro. Al suo posto, è stato implementato un sistema di hashing basato sull'algoritmo **SHA-256** (*Secure Hash Algorithm 256-bit*).

Ogni volta che un utente inserisce una password (in registrazione o al login), il metodo **hashPassword()** la trasforma in un'impronta digitale univoca formata da 64 caratteri esadecimali. L'unidirezionalità di questo processo rende computazionalmente impossibile risalire alla password originale partendo dall'hash. In questo modo, qualora il file **FauxTo.db** venisse impropriamente estratto dal dispositivo, le credenziali rimarrebbero protette, elevando il livello di sicurezza generale.

Per implementare questa funzionalità sono state utilizzate due classi del package **java.security**:

- **MessageDigest**: genera l'hash della password. L'istruzione **getInstance('SHA-256')** inizializza l'algoritmo, mentre **digest()** produce l'hash.
- **NoSuchAlgorithmException**: un'eccezione, gestita tramite un blocco **try-catch**, che si verifica qualora l'algoritmo non sia supportato dal sistema.

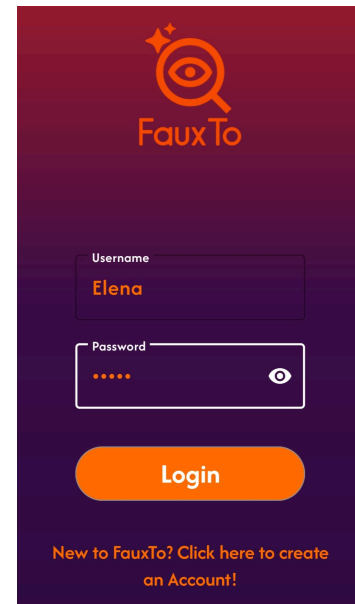
Il metodo **hashPassword()** converte infine l'array di byte prodotto dal digest in una stringa esadecimale, più agevole da salvare e confrontare. Al momento dell'accesso, la password digitata viene hashata nuovamente e confrontata con la stringa sicura conservata nel database.



## 3 Activity di Login

La `LoginActivity` rappresenta il punto di ingresso dell'applicazione e gestisce l'autenticazione dell'utente. L'Activity adotta il design *Edge-to-Edge*, estendendo l'interfaccia grafica sotto le barre di sistema (stato e navigazione) e gestendo i margini di sicurezza tramite l'oggetto `WindowInsetsCompat`. L'interfaccia presenta i seguenti elementi:

- Una `ImageView` con il logo dell'applicazione;
- Due campi `TextInput` per inserire username e password;
- Un bottone "Login" per confermare le credenziali;
- Una `TextView` con il testo "New to FauxTo? Click here..." per reindirizzare alla schermata di registrazione.



### 3.1 Flusso di autenticazione

Alla pressione del pulsante Login, il sistema esegue i seguenti passaggi:

1. **Controllo campi vuoti:** se i campi username o password sono lasciati vuoti, viene mostrato un messaggio di avviso tramite un `Toast`.
2. **Validazione credenziali:** se i campi sono compilati, il metodo `checkUser()` della classe `DatabaseHelper` verifica la corrispondenza della coppia username-password nel database. La password inserita viene trasformata in hash per il confronto.

```
if (username.isEmpty() || password.isEmpty()) {  
    Toast.makeText(LoginActivity.this, R.string.error_login_fields,  
        Toast.LENGTH_SHORT).show(); }  

```

```
if (dbHelper.checkUser(username, password))
```

3. **Accesso:** se la validazione ha successo, un `Intent` avvia l'Activity `ImageViewer` e la `LoginActivity` viene chiusa con `finish()`, impedendo all'utente di tornare alla schermata di login con il tasto back, migliorando l'esperienza utente ed evitando che un utente già autenticato debba reinserire le credenziali per errore.

```
Intent intent = new Intent(LoginActivity.this, ImageViewer.class);  
startActivity(intent);  
finish();
```

4. **Errore:** se le credenziali non sono corrette, viene mostrato un messaggio tramite un `Toast` "Wrong Username or Password".

```
Toast.makeText(LoginActivity.this,  
    R.string.login_error, Toast.LENGTH_SHORT).show();
```

### 3.2 Passaggio alla Registrazione

La scritta "New to FauxTo? Click here..." funge da link verso la `RegisterActivity`. Alla pressione, un `Intent` avvia la schermata di registrazione. Trattandosi di una `TextView` e non di un pulsante, sono stati impostati gli attributi `clickable` e `focusable` a `true` nel file XML per abilitarne l'interazione.

### 3.3 Splash Screen

La `LoginActivity` implementa la Splash Screen ufficiale di Android tramite la libreria `androidx.core:core-splashscreen`. Il funzionamento si articola in tre fasi:

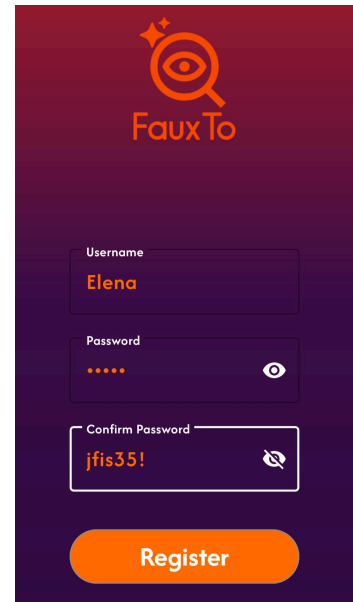
1. **Definizione del tema:** nel file `themes.xml` è stato definito lo stile `Theme.App.Starting`, che eredita da `Theme.SplashScreen` e specifica:
  - `windowSplashScreenBackground`: colore di sfondo durante il caricamento;
  - `windowSplashScreenAnimatedIcon`: icona del logo visualizzata al centro;
  - `postSplashScreenTheme`: tema applicativo (`Theme.FauxTo`) da applicare al termine dell'animazione.
2. **Configurazione nel Manifest:** nel file `AndroidManifest.xml`, la `LoginActivity` è dichiarata come punto di ingresso tramite l'`intent-filter` (azioni `MAIN` e `LAUNCHER`) e utilizza il tema `@style/Theme.App.Starting`, permettendo al sistema di visualizzare lo splash screen prima ancora dell'avvio del processo Java.
3. **Inizializzazione nel codice:** nel metodo `onCreate()` della `LoginActivity`, la chiamata a `SplashScreen.installSplashScreen(this)` viene eseguita prima di `super.onCreate()`, gestendo la transizione fluida verso l'interfaccia di login.

Questo approccio garantisce che lo splash screen appaia istantaneamente al tocco dell'icona, offrendo un'esperienza di avvio reattiva prima del caricamento completo del codice applicativo.

## 4 Activity di Registrazione

La `RegisterActivity` gestisce la creazione di un nuovo account utente. Anche questa Activity implementa il design *Edge-to-Edge*; per garantire che i campi di input non vengano coperti dalla tastiera o dalle barre di navigazione, è stato implementato un listener che gestisce dinamicamente i *WindowInsets*, adattando l'interfaccia alle dimensioni dello schermo. L'interfaccia presenta:

- Una `ImageView` con il logo dell'applicazione;
- Tre campi `TextInput` per username, password e conferma password;
- Un bottone "Register" per finalizzare la creazione dell'account.



### 4.1 Flusso di autenticazione

Alla pressione del pulsante Register, il sistema esegue una serie di quattro controlli sequenziali:

1. **Campi vuoti:** se l'utente lascia vuoto anche solo uno dei tre campi, un `Toast` notifica l'errore.

```
if (username.isEmpty() || password.isEmpty() || passwordConfirm.isEmpty()) {
    Toast.makeText(this, R.string.error_fill_all, Toast.LENGTH_SHORT).show(); }
```

2. **Corrispondenza password:** il sistema confronta le due password. In caso di discrepanza, il metodo `setError()` visualizza un messaggio di errore contestuale sul campo di conferma.

```
else if (!password.equals(passwordConfirm)) {
    passwordConfirmInput.setError(getString(R.string.error_password_mismatch)); }
```

3. **Verifica username:** tramite il metodo `checkUserExists()` di `DatabaseHelper`, il sistema controlla se lo username è già registrato. Anche in questo caso si utilizza `setError()`, fornendo un feedback immediato e non invasivo che indica esattamente quale campo necessita di correzione.

```
else if (dbHelper.checkUserExists(username)) {
    usernameInput.setError(getString(R.string.error_user_exists)); }
```

4. **Registrazione:** se tutti i test sono superati, il metodo `addUser()` salva le credenziali (previa trasformazione in hash della password) nel database e l'Activity viene chiusa con `finish()`, riportando l'utente al Login.

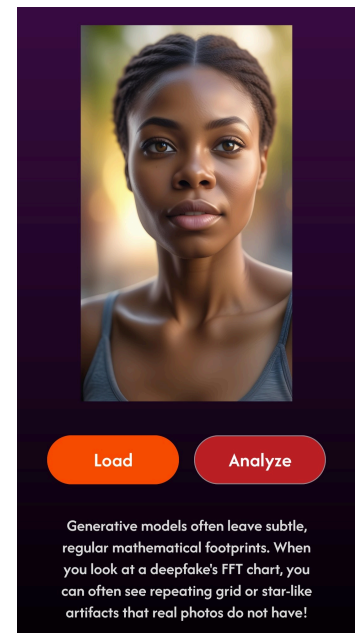
```
else {  
    dbHelper.addUser(username, password);  
    finish(); }  
}
```

La chiamata al metodo `finish()` non solo libera le risorse di memoria, ma agisce come segnale di completamento, guidando l'utente verso la schermata di autenticazione per l'accesso immediato. Infine, nel file di layout `activity_register.xml`, per i campi relativi alle password è stato impostato l'attributo `android:inputType="textPassword"`. Questo accorgimento garantisce che i caratteri digitati siano mascherati, proteggendo la privacy dell'utente in conformità con i requisiti di sicurezza delle interfacce mobili.

## 5 Activity Image Viewer

L'`ImageViewer` costituisce la schermata principale dell'applicazione, dove l'utente carica l'immagine da analizzare e avvia il processo di rilevamento. L'Activity adotta un design **Edge-to-Edge**, estendendo il background (`gradient.black.xml`) fin sotto le barre di sistema. L'interfaccia presenta:

- Una `ImageView` centrale per l'anteprima dell'immagine;
- Un bottone "Load" per il caricamento da galleria;
- Un bottone "Analyze" per l'avvio del processo;
- Una `ProgressBar` circolare per il feedback durante l'elaborazione.



### 5.1 Caricamento dell'immagine e Gestione dei permessi

L'accesso alle immagini salvate sul dispositivo richiede permessi specifici, gestiti dinamicamente per garantire compatibilità con le diverse versioni di Android. I permessi sono dichiarati nel file `AndroidManifest.xml`:

```
<uses-permission android:name="android.permission.READ_MEDIA_IMAGES" />  
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"  
    android:maxSdkVersion="32" />
```

La distinzione tra le direttive riflette l'evoluzione delle API: `READ_MEDIA_IMAGES` è richiesto da Android 13 in poi, mentre `READ_EXTERNAL_STORAGE` è mantenuto esclusivamente per le versioni legacy tramite l'attributo `android:maxSdkVersion`.

Il metodo `checkPermissionAndLaunchPicker()`, gestisce la richiesta:

```
private void checkPermissionAndLaunchPicker() {  
    String permission;  
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {  
        permission = Manifest.permission.READ_MEDIA_IMAGES;  
    } else {  
        permission = Manifest.permission.READ_EXTERNAL_STORAGE;  
    }  
}
```

Se il permesso risulta già concesso, l'applicazione apre direttamente il file manager di sistema tramite il contratto moderno `ActivityResultContracts.GetContent`, filtrando esclusivamente immagini in formato JPEG. In caso contrario, viene mostrato il popup nativo di richiesta del consenso; qualora l'utente neghi l'autorizzazione, l'app intercetta l'evento mostrando un messaggio di avviso tramite `Toast`.

Una volta ottenuto l'accesso alla galleria, l'utente può selezionare il file desiderato, che viene associato e visualizzato nell'`ImageView` tramite il relativo URI. Questo approccio evita il caricamento dell'immagine a piena risoluzione in memoria, riducendo il consumo di risorse e migliorando l'efficienza complessiva dell'applicazione.

## 5.2 Analisi (Background Thread)

Per evitare il blocco dell'interfaccia utente (UI Freeze) durante i calcoli intensivi della FFT e dell'inferenza ONNX, la gestione del lavoro è affidata a un `ExecutorService` con un singolo thread. Questo permette di accodare le richieste di analisi ed eseguirle in background, mantenendo l'applicazione fluida e reattiva. Sul thread secondario avvengono tre operazioni fondamentali:

1. **Caricamento del Bitmap:** il metodo `loadBitmap()` converte l'URI in un oggetto `Bitmap`. Su API 28+ si utilizza `ImageDecoder`, più efficiente rispetto al classico `MediaStore.Images.Media.getBitmap()`.
2. **Inferenza del modello:** il `Bitmap` è passato a `DeepfakeDetector.getDeepfakeProbability()`, che esegue:
  - Ridimensionamento a  $224 \times 224$  pixel.
  - Calcolo della FFT su tre canali RGB.
  - Inferenza tramite modello ONNX su `ONNX Runtime`.
3. **Generazione della heatmap FFT:** il metodo `generateFFTPreview()` produce una visualizzazione dello spettro tramite la libreria *JTransforms*. L'immagine è convertita in scala di grigi, si applica la FFT 2D, il logaritmo per il range dinamico e una palette cromatica (blu-arancione), integrando una correzione gamma per enfatizzare gli artefatti rispetto al rumore.

## 5.3 Salvataggio e passaggio dati

Per evitare crash dovuti a limiti di memoria nel passaggio di `Bitmap` pesanti tramite `Intent`, l'anteprima viene salvata come `result_preview.png` nella cache interna. Viene passato alla `ResultActivity` solo il path del file e la percentuale di probabilità.

Al termine, `runOnUiThread()` ripristina l'interazione con l'UI, nascondendo la `ProgressBar`. Per garantire un uso efficiente della memoria RAM, l'applicazione invoca il metodo `recycle()` sulle `Bitmap` non più necessarie. Inoltre, nel metodo `onDestroy()` dell'`Activity`, l'`ExecutorService` viene arrestato e la sessione ONNX viene chiusa per liberare le risorse native allocate.

# 6 Addestramento e Conversione del Modello ONNX

Data la scarsità di modelli ONNX pre-addestrati già pronti per il riconoscimento di deepfake tramite analisi FFT, è stato necessario addestrare una rete neurale da zero e convertirla in formato ONNX per l'integrazione su Android.

## 6.1 Dataset

L'addestramento è avvenuto su Google Colab con il dataset pubblico *140k Real and Fake Faces*. Sono stati selezionati 400 campioni (200 reali, 200 sintetici), ridimensionati a  $224 \times 224$  e normalizzati in  $[0, 1]$ , calcolando la FFT canale per canale.

- **Ramo spaziale:** utilizza MobileNetV2<sup>3</sup> pre-addestrato su ImageNet<sup>4</sup> come backbone per l'estrazione di feature visive. I primi 150 layer sono congelati (transfer learning): questo permette di mantenere la capacità di riconoscere forme, bordi e texture appresa su milioni di immagini generiche. I layer superiori vengono invece riaddestrati sul nostro dataset di volti reali e sintetici per specializzare il modello sul task specifico.
- **Ramo frequenziale:** una CNN compatta elabora lo spettro FFT dell'immagine, calcolato canale per canale e normalizzato. Questo ramo è progettato per catturare gli artefatti periodici caratteristici delle immagini generate, che appaiono come pattern anomali nello spettro delle frequenze.

---

<sup>3</sup>MobileNetV2 è una rete neurale convoluzionale (CNN) leggera, ottimizzata per l'uso su dispositivi mobili e sistemi embedded con risorse computazionali limitate.

<sup>4</sup>ImageNet è uno dei più grandi database di immagini al mondo, contenente oltre 14 milioni di immagini annotate in 20.000 categorie.

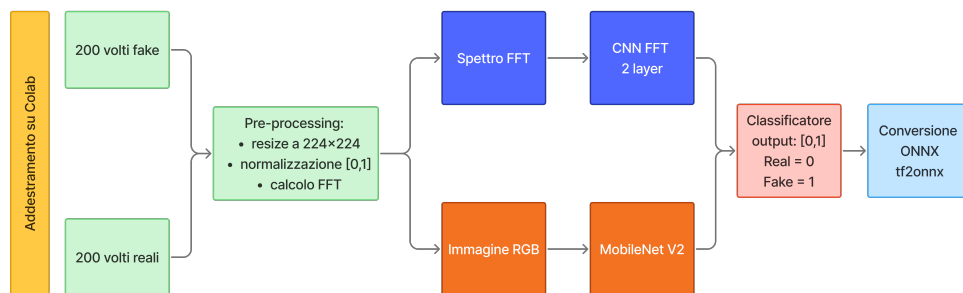
I due rami operano in parallelo: MobileNetV2 analizza l'aspetto visivo dell'immagine (pixel), mentre la CNN frequenziale esamina lo spettro FFT alla ricerca di artefatti. I risultati, concatenati, alimentano un classificatore finale che restituisce una probabilità  $[0, 1]$ .

## 6.2 Conversione in ONNX

Al termine dell'addestramento, il modello TensorFlow/Keras è stato convertito in formato ONNX tramite la libreria `tf2onnx`. Questa conversione è necessaria perché ONNX Runtime, utilizzato su Android, non può eseguire direttamente modelli TensorFlow. Il modello convertito ha due input, entrambi di forma  $[1, 224, 224, 3]$ :

- Il primo numero (1) rappresenta il batch size, cioè quante immagini vengono analizzate contemporaneamente. Nell'app viene analizzata un'immagine alla volta.
- $224 \times 224$  sono le dimensioni in pixel richieste dal modello.
- 3 sono i canali di colore RGB (Red, Green, Blue).

L'output è un singolo valore scalare tra 0 e 1, che rappresenta la probabilità che l'immagine sia un deepfake.



### 6.2.1 Codice per l'Addestramento su Colab

Di seguito viene riportata una versione ridotta del codice usato su Colab, generato con il Large Language Model Deepseek.

```
# 1. Installa le librerie necessarie (TensorFlow per il deep
learning, tf2onnx per la conversione in ONNX) e scarica il dataset di volti
reali e sintetici.
```

```
!pip install tensorflow tf2onnx onnx kagglehub -q
path = kagglehub.dataset_download("xhlulu/140k-real-and-fake-faces")
```

```
# 2. Definisce due funzioni. La prima carica un'immagine, la
ridimensiona a 224x224 pixel e normalizza i valori tra 0 e 1. La seconda
calcola la Trasformata di Fourier (FFT) dell'immagine: scompone ciascun
canale RGB nelle sue frequenze, applica lo shift per centrare le basse
frequenze, calcola la magnitudo e la comprime con il logaritmo, infine
normalizza tutto tra 0 e 1.
```

```
img = Image.open(file_path).convert('RGB').resize((224, 224))
return np.array(img).astype(np.float32) / 255.0

def calculate_fft(images):
    # FFT 2D canale per canale, shift, magnitudo, log, normalizzazione
    ...
```

```
# 3. Carica 200 immagini reali e 200 immagini fake, per un
totale di 400 campioni di addestramento.
```

```
X_real = np.array([load_and_preprocess(f) for f in real_files[:200]])
X_fake = np.array([load_and_preprocess(f) for f in fake_files[:200]])
```

```
# 4. Costruisce il modello dual-branch. Il ramo spaziale usa
```



MobileNetV2 (già addestrata su milioni di immagini) per analizzare i pixel. Il ramo frequenziale usa una piccola CNN per analizzare lo spettro FFT. I risultati dei due rami vengono uniti e passati a un classificatore che produce un numero tra 0 (immagine reale) e 1 (deepfake).

```
input_spatial = keras.Input(shape=(224, 224, 3), name="image")
base_model = keras.applications.MobileNetV2(weights='imagenet', ...)

input_frequency = keras.Input(shape=(224, 224, 3), name="fft_image")
y = layers.Conv2D(32, 3, ...) # Ramo FFT (2 layer: 32 e 64 filtri)

combined = layers.concatenate([x, y]) # Unione dei due rami
output = layers.Dense(1, activation='sigmoid')(combined)

# 5. Addestra il modello per 10 epoche, elaborando 16
immagini alla volta. L'ottimizzatore Adam aggiusta i pesi della rete per
minimizzare l'errore tra predizioni e realtà.

model.compile(optimizer=keras.optimizers.Adam(1e-4), ...)
model.fit([X_train, X_train_fft], y_train, epochs=10, batch_size=16)

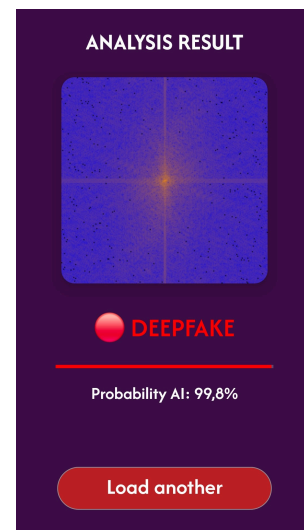
# 6. Converti il modello TensorFlow in formato ONNX,
rendendolo utilizzabile su Android tramite ONNX Runtime.

tf2onnx.convert.from_keras(model, ...)
onnx.save(model_proto, "deepfake_improved.onnx")
```

## 7 Result Activity

La `ResultActivity` è la schermata finale che presenta l'esito dell'analisi. Riceve i dati dall'`ImageViewer` tramite `Intent` e adatta dinamicamente l'interfaccia in base al verdetto [cite: 262]. L'interfaccia include:

- Un titolo “ANALYSIS RESULT”;
- Una `CardView` contenente l'heatmap FFT generata;
- Una `TextView` per il verdetto testuale (es. “DEEPFAKE DETECTED”);
- Una `ProgressBar` e una `TextView` per la percentuale di confidenza;
- Un bottone “Load another image” per riavviare il processo [cite: 263-269].



### 7.1 Inizializzazione

Anche questa `Activity` utilizza la modalità `EdgeToEdge` per estendere l'interfaccia fino ai bordi dello schermo. I componenti UI vengono inizializzati nel metodo `onCreate()` tramite i relativi ID.

```
EdgeToEdge.enable(this);
super.setContentView(R.layout.activity_result);
```

### 7.2 Ricezione dei Dati tramite Intent

L'`Activity` riceve i risultati dall'`ImageViewer` tramite un oggetto `Intent`, evitando calcoli autonomi.

I dati trasmessi includono la probabilità calcolata dal modello e il percorso del file dell'heatmap:

```
double aiProbability = getIntent().getDoubleExtra("percentage", 0.0);
String imagePath = getIntent().getStringExtra("imagePath");
```

- **percentage**: un valore `double` che rappresenta la probabilità calcolata dal modello ONNX. Il secondo parametro (0.0) è il valore di default restituito se la chiave non viene trovata, evitando così crash dell'app.
- **imagePath**: una stringa con il percorso del file PNG salvato nella cache dell'app, contenente l'anteprima della heatmap FFT.

### 7.3 Logica di Classificazione a Soglie

L'applicazione confronta la percentuale di probabilità con quattro soglie predefinite, aggiornando dinamicamente il testo e il colore della UI:

```
if (aiProbability > 70) {
    tvClassification.setText(R.string.label_deepfake);
    tvClassification.setTextColor(Color.RED);
} else if (aiProbability > 50) {
    tvClassification.setText(R.string.label_possible);
    tvClassification.setTextColor(Color.parseColor("#FF9800"));
} else if (aiProbability > 30) {
    tvClassification.setText(R.string.label_uncertain);
    tvClassification.setTextColor(Color.parseColor("#FFC107"));
} else {
    tvClassification.setText(R.string.label_authentic);
    tvClassification.setTextColor(Color.GREEN);}
```

Le quattro soglie sono:

- **Sopra il 70% (Rosso)**: l'immagine è quasi certamente un deepfake. Si usa la costante predefinita `Color.RED`.
- **Sopra il 50% (Arancione)**: probabile manipolazione AI. Il colore viene creato dal codice esadecimale `#FF9800` tramite `Color.parseColor()`.
- **Sopra il 30% (Giallo)**: risultato incerto, sono stati rilevati pattern sospetti ma non decisivi, anche in questo caso il colore viene creato tramite `Color.parseColor()`.
- **Sotto il 30% (Verde)**: l'immagine è considerata autentica. Si usa la costante predefinita (`Color.GREEN`).

Oltre al testo, il codice modifica il colore della `ProgressBar` tramite `setProgressTintList()`. Questo metodo richiede un `ColorStateList` per applicare correttamente la colorazione dinamica (Rosso, Arancione, Giallo, Verde) al tracciato della barra, fornendo un feedback visivo immediato all'utente:

```
pbConfidence.setProgressTintList(
    android.content.res.ColorStateList.valueOf(Color.RED));
```

### 7.4 Visualizzazione dello Spettro FFT

L'Activity recupera e visualizza l'immagine FFT precedentemente salvata nella cache e la mostra all'interno di una `CardView`:

```
if (imagePath != null) {
    Bitmap bitmap = BitmapFactory.decodeFile(imagePath);
    ivSpectrum.setImageBitmap(bitmap); }
```

Lo scopo di questa visualizzazione è fornire all'utente una prova visiva dell'analisi effettuata. Anche senza conoscenze tecniche, l'utente può osservare pattern geometrici o punti luminosi anomali nello spettro e capire che il verdetto si basa su dati matematici reali (gli artefatti di frequenza) e non su un numero casuale.

Il controllo `if (imagePath != null)` previene errori nel caso in cui il percorso non sia stato passato correttamente. `BitmapFactory.decodeFile()` legge il file PNG dalla cache e lo converte in un oggetto `Bitmap`, che viene poi mostrato nella `ImageView`.

Anche in questa schermata, tutti i testi sono estratti dal file `strings.xml`. Questo permette di mantenere la logica Java pulita e focalizzata solo sul calcolo delle soglie, delegando la gestione delle etichette testuali alle risorse esterne dell'app.